200+ AI System Design Interview Questions

Clear your interviews with confidence

You've been asked to add an AI feature to an existing product, but the PM only says "make it smarter." Walk through every clarifying question you would ask before writing a single line of design.

"Make it smarter" is a product wish, not a requirement. Before designing anything, I need to convert that wish into a concrete problem statement with measurable outcomes. Here's exactly how I'd do that.

Questions about the problem:

- What specific user behavior or pain point is driving this request? What are users currently doing that they shouldn't have to do manually?
- What does "smarter" mean in this context? Faster responses? More accurate results? Personalized recommendations? These are completely different systems.
- How are users currently solving this problem? Are they working around a limitation, or is this net-new capability?

Questions about users and usage:

- Who are the primary users? Are they technical or non-technical?
- How often will they interact with this feature? Real-time queries, daily batch, or event-triggered?
- What does a successful interaction look like from the user's perspective? Can you show me an example of a "good" output?

Questions about data and quality:

- What data does the system have access to? Is it clean and labeled?
- How do we measure success? Is there a baseline we're trying to beat?
- What's the error tolerance? If the model is wrong 5% of the time, is that acceptable or catastrophic?

Getting answers to these before writing a single line saves weeks of rework. The biggest design mistakes I've seen all trace back to assumptions made at this stage.

You're given the goal: "improve user experience using AI." How do you break this down into a concrete, scoped system with measurable outcomes?

"Improve user experience using AI" is a direction, not a specification. My job is to decompose it until it becomes buildable and testable.

Step 1: Identify the friction point
I'd start by finding the specific moment in the user journey where the experience breaks down. Is it that users can't find what they're looking for? Are they doing repetitive work? You can't improve user experience in general. You can improve a specific step in a specific workflow.

Step 2: Translate the UX goal into an AI task type

- Users can't find relevant content: this is a **retrieval** problem.
- Users are getting one-size-fits-all responses: this is a **personalization** problem.
- Users are doing repetitive data entry: this is an **extraction or automation** problem.

The task type determines the system architecture entirely.

Step 3: Define measurable outcomes
For each task type, I'd define at least one leading metric (measures system output quality) and one lagging metric (measures actual user impact).

Step 4: Define what "done" looks like
Set a specific target: "reduce average time-to-answer from 3 minutes to under 45 seconds." A target without a number isn't a target.

Step 5: Scope the MVP
Define the narrowest version of this that delivers measurable value. Pick one user type, one workflow, one data source. Get a number. Expand from there.

What You'll Cover Inside

VOLUME 1

Foundations

System thinking, trade-offs

VOLUME 2

LLM Systems

Prompts, context, pipelines

VOLUME 3

RAG Systems

Retrieval, ranking, optimization

VOLUME 4

Agent Systems

Workflows, tools, memory

VOLUME 5

Production Systems

Scaling, latency, cost

VOLUME 6

Real Scenarios

ChatGPT, RAG, voice AI

Check caption for Interview Kit

A traditional backend engineer joins your AI team. What are the three biggest mindset shifts they need to make?

Backend engineers are excellent at building reliable, deterministic systems. AI systems are fundamentally different.

Shift 1: From determinism to probabilistic thinking
In traditional backend systems, the same input produces the same output. In AI systems, output varies. Quality is measured statistically, not absolutely. You stop thinking about testing as "pass/fail" and start measuring "what fraction of the time does this produce acceptable output."

Shift 2: From code correctness to data quality
Backend engineers find bugs in code. In AI systems, the code is often fine. The problem is the data: biased training data, stale retrieval data, misrepresentative evaluation data. Investing heavily in data pipelines and evaluation datasets is where actual quality work happens here.

Shift 3: From binary pass/fail to graceful degradation
In traditional systems, if something is wrong you fix it. In AI systems, you design for the fact that the model will sometimes be wrong or unreliable. You're not building a system that's always right. You're building a system that fails safely and recovers intelligently.

Check caption for Interview Kit

Your AI system produces slightly different outputs for the same input. How does this non-determinism affect downstream components?

Non-determinism isn't a bug you can fix. It's a property of the system you have to design around.

Components that break without careful handling:

- **Caching layers:** Traditional caches assume same input = same output. Caching LLM responses guarantees variance reduction, which might mean caching bad responses.
- **Deterministic testing:** Test suites asserting exact string matches will fail constantly. You need semantic assertions or a separate evaluator model.
- **Downstream integrations:** If an AI system routes customers differently run-to-run, downstream queues need to handle inconsistency.
- **Audit and logging:** Logs for the same input at two different times will show different content.

Practical design responses:

- Set temperature to 0 for system paths requiring determinism (safety checks, classification).
- Log full context with every response to understand output provenance.
- Design downstream systems to be tolerant of output variance within an acceptable range.

Check caption for Interview Kit

After a model upgrade, accuracy improves by 15% but p99 latency doubles. How do you decide what to prioritize?

This is a classic engineering trade-off where the right answer depends on who is waiting for the response.

The real-time chat interface:
Users are waiting. If p99 latency crosses the threshold where users perceive a lag (generally >1-2 seconds), you have a UX problem. Users notice slowness immediately; they rarely notice direct 15% accuracy bumps. I'd keep the old model until I can close the latency gap via streaming tokens or caching.

The batch reporting pipeline:
Latency barely matters async. If a job runs overnight and takes 20% longer, that's fine. The 15% accuracy improvement, however, directly impacts report quality and business decisions. I'd upgrade immediately here.

The general principle:
Route the two use cases to different model versions. Serving different models to different pipelines lets you optimize each independently. The infrastructure complexity is worth it because the trade-offs are genuinely different.

You're launching an AI feature in 4 weeks. How do you define success metrics before launch?

Metrics defined after launch are justified, not measured. You need them defined before so you know what data to collect.

Pre-launch metric definition:
For each metric, define what exactly you measure, how you collect data instrumentally, the baseline, and the success condition at 30, 60, and 90 days.

Typical categories for AI features:

- **Quality:** Task completion rate, user correction rate (how often users reject output).
- **Engagement:** Feature adoption rate, retention rate.
- **Business:** Downstream conversion rate change, support ticket deflection rate.

How they evolve post-launch:
Weeks 1-2 focus on catastrophic failures (error rates, security). Weeks 3-4 shift to optimizing leading metrics like task completion. Month 2+ focuses on lagging business metrics like revenue impact.

A junior engineer defines requirements as: "the model should be accurate and fast." What's wrong with this?

"Accurate and fast" is the equivalent of "make it good." It provides no actionable specification.

"Accurate" is undefined:
Accurate on what data? Measured how (Precision/Recall/F1)? Compared to what baseline? Acceptable at what threshold (80% or 99.9%)?

"Fast" is equally vague:
Fast for the median user or the 99th percentile? Under what load? Fast at what stage (Inference only or end-to-end)? What's the upper bound in milliseconds?

How I'd guide them:
Rewrite requirements by answering three questions:
1. How will we measure this in production?
2. What's the specific threshold that defines pass/fail?
3. Under what conditions (load, data type)?

Better version: "The system must return a response with an F1 score of at least 0.85 on our evaluation set, measured weekly, at p95 latency under 800ms at 500 concurrent users."

Your system must respond in under 150ms end-to-end. Walk through how this constraint shapes design.

150ms end-to-end eliminates a large number of common architectural choices instantly.

Budget allocation: 10ms preprocessing, 20ms retrieval, 100ms inference, 15ms postprocessing, 5ms network overhead.

- **Model choice:** Large models (70B+) and shared GPT-4 endpoints are off the table. You need small, fast models (7B-13B) on dedicated, high-performance inference infrastructure.
- **Retrieval constraints:** Full vector search adds 50-200ms. At a 150ms budget, you must use in-memory caches, pre-computed results, or drastically limit the search scope.
- **Prompt length:** Long prompts increase processing time. Keep prompts minimal, pre-tokenize, and cache them at the inference layer.
- **Output validation:** No synchronous LLM-based validation. It has to be lightweight (Regex/Schema) or async.

The honest answer: This is possible but requires trade-offs—small fast models, no complex retrieval. If the PM wants 150ms AND high quality AND full RAG, something has to give.

Under load, one pipeline component will bottleneck first. How do you identify which one before production?

The component with the lowest throughput relative to demand bottlenecks first.

Step 1: Independent Latency Profiling
Run synthetic tests on each stage isolated. Embedding generation might be 10-50ms. Retrieval: 20-100ms. LLM Inference: 100ms-2s. Inference is almost always the bottleneck as it's compute-heavy and rarely scales linearly.

Step 2: Mathematical Modeling
Calculate: if targeting 100 RPS and inference takes 500ms, you need 50 concurrent slots to keep up. If your GPU handles 10 concurrent requests, you're 5x under before even deploying.

Step 3: Realistic Load Testing
Run tests with gradual ramp-ups, monitoring queue depth (not just latency). A growing queue reveals the bottleneck instantly.

Step 4: Pre-deployment Monitoring
Set alerts on component-specific metrics: request rates, p99 latency, queue depth, and resource utilization. Spot the stress before users do.

How does explicitly mapping data flow at each stage change the quality of your design?

Mapping data flow is not a documentation exercise. It's a design tool that reveals problems before you build.

What it forces you to define:

- What is the exact schema entering this stage?
- What transformation happens inside?
- What happens when it fails?

Most architectures fail at the stage transitions. If preprocessing outputs a raw string but retrieval expects a JSON query object, you've found a contract violation early.

Concrete improvements:

- **Explicit Contracts:** Constrains changes without breaking downstream components.
- **Visible Error Handling:** If retrieval returns zero results, does generation get a null, empty list, or error?
- **Predictable Bottlenecks:** You can estimate compute cost instantly via data sizing.
- **Granular Testing:** Enables robust unit testing for individual components.

You've been asked to design a system where the core logic is "call GPT-4 and return the result." Why is this not a system design?

Calling the API is a function call. A system design is everything else that makes that API call production-worthy.

What's actually missing:

- **Input Handling:** Handling multiple languages, preventing prompt injections, sanitizing text limits.
- **Context Management:** Keeping multi-turn conversation history within API token limits via pruning or summarization.
- **Retrieval (RAG):** If factual accuracy is needed, you need an index for ingestion, embedding, storage, and querying.
- **Reliability:** API timeouts, rate limit backoffs, handling model refusals and JSON schema malformations.
- **Cost & Observability:** Caching mechanisms, structured logging to debug specific bad responses.

The core logic is merely the engine. System design is the transmission, steering, brakes, and chassis.

Check caption for Interview Kit

You need to design a system for legal document review. What are the boundaries and contracts?

Legal AI has high stakes. The critical boundary: The system outputs risk findings with attribution, not final legal conclusions.

Document Ingestion:
Contract: Every document produces either a normalized DB record or a clear format error. Maintains pagination mapping.

Chunking and Embedding:
Contract: Every chunk maintains a traceable reference vector to its source document and page location.

Clause Classification:
Contract: Extracts and tags clauses (e.g., Liability, Indemnification). Evaluates confidence scores. Every extracted node links directly back to original text.

Risk Analysis:
Contract: Compares extracted clauses against corporate playbooks. Flags deviations. Produces risk findings citing specific clauses.

The Final Contract: Everything presented to the attorney interface must highlight explicitly that human review is required, protecting both user trust and legal liability.

Check caption for Interview Kit

Your system has retrieval, orchestration, and generation. Due to a budget cut, one must be simplified. Trade-off strategy?

Simplifying Retrieval:
Generation relies heavily on parametric knowledge. Acceptable if the domain is well-covered natively and freshness doesn't matter. Unacceptable for internal enterprise Q&A due to hallucination risks.

Simplifying Orchestration:
Removes complex routing and multi-step reasoning. Fine for simple Q&A. Fatal for complex Agentic workflows needing tool use and conditionals.

Simplifying Generation:
Replacing massive LLMs with templates or smaller models rigidly restricts natural language capability. The system loses its synthesizing value proposition.

Recommendation:
Simplify retrieval before touching generation, unless precise internal info is the core product. A smart generator with simple DB lookups often beats a rigidly templated generator with ultra-sophisticated vector search. Quality perception relies on generation.

Check caption for Interview Kit

Your prototype works flawlessly in a notebook. List assumptions it makes which production will invalidate.

Notebooks are built to work. Production is built to survive.

Data assumptions:
Notebook uses a clean sample. Production receives malformed data, wild character encodings, and edge cases exceeding context windows.

Latency & Scale:
Notebook processes sequentially on local hardware. Production faces concurrent spikes under shared infrastructure limits and cold-start model latency jumps.

Reliability:
Notebook assumes 100% API uptime. Production faces rate limits, timeouts, and partial pipeline failures without retry logic.

Quality & Security:
Notebook answers were verified visually on friendly inputs. Production will see adversarial prompt injections relying entirely on unwritten validation logic. Without telemetry, production flies blind.

Check caption for Interview Kit

You have a $500/month infrastructure budget to serve 10,000 users. Architect this cheaply.

Usage Math first:
If 2,000 are daily active users doing 5 queries = 10,000 queries/day. 300K queries/month. That's $0.0016 per query budget.

Model Selection limits:
Tokens for standard GPT-4 will bankrupt this in days.
Opt for ultra-efficient API providers (like Claude Haiku or GPT-3.5) keeping costs sub-$50. Alternatively, small OSS model instances scaling to zero during zero traffic.

Caching heavily:
Cache effectively. Use semantic caching strategies for repeat/similar queries; an 80% hit-rate translates directly to pure margin.

Prompt Efficiency:
Shorten instructions dramatically. Limit max-tokens returned. Do not run LLM validation queries—use regex instead. Batch any background asynchronous jobs aggressively during off-peak times to leverage cheaper tiers.

Check caption for Interview Kit

200+ Questions Like This

This is just a small sample.

INSIDE THE FULL KIT:

- → 200+ real interview questions
- → Clear, structured answers
- → System design frameworks

If you're serious about AI system design interviews, check the caption for Interview Kit link.